

OPERATING SYSTEMS
UE25CS242A

Multicore Kernel Concepts

Dr. Anitha K
Department of Computer Science and Engineering

Acknowledgement, Prof. Pavan A C

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:

1. "Operating System Concepts", Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. "Advanced Programming in the Unix Environment", Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from "Operating Systems, Internals and Design Principles", William Stallings, 9th Edition, Pearson, 2018.

Operating Systems

Multicore Kernel Concepts

Dr. Anitha K
Department of Computer Science and Engineering

Why Multi-Processor Scheduling?

PES UNIVERSITY
CELEBRATING 50 YEARS

- Everything so far assumed a **single processing core** — one thread runs at a time.
- With multiple CPUs we achieve **load sharing**: multiple threads may run in parallel.
- But the scheduling problem becomes increasingly complex- **there is no single best solution.**
- Traditionally multiprocessor meant several physical single-core chips. Today the term covers:
  - **Multicore CPUs** — several cores on one chip
  - **Multithreaded cores** — several hardware threads per core
  - **NUMA systems** — memory access cost depends on which CPU
  - **Heterogeneous multiprocessing** — cores of differing capability
- Approaches to Multiple-Processor Scheduling:
  - **Symmetric multiprocessing**

Asymmetric Multiprocessing



- One processor—the **master server**—handles all scheduling decisions, I/O processing and other system activities.
- All other processors **execute user code only**.
- **Simple**, because only one core touches the system data structures—far less data sharing is needed.
- **Disadvantage:** the master server becomes a potential **bottleneck**, so overall system performance can drop.

Symmetric Multiprocessing (SMP)



- The **standard approach** for supporting multiprocessors.
- Each processor is **self-scheduling**: its own scheduler examines a ready queue and picks a thread to run.
- **No master server** — system and user code can run on any core.
- Two possible strategies for organizing the threads:
  - 1. All threads may be in a common ready queue, or
  - 2. Each processor may keep its own private queue of threads
- Virtually all modern operating systems support SMP — Windows, Linux, macOS, and mobile systems including Android and iOS.

Organization of Ready Queues

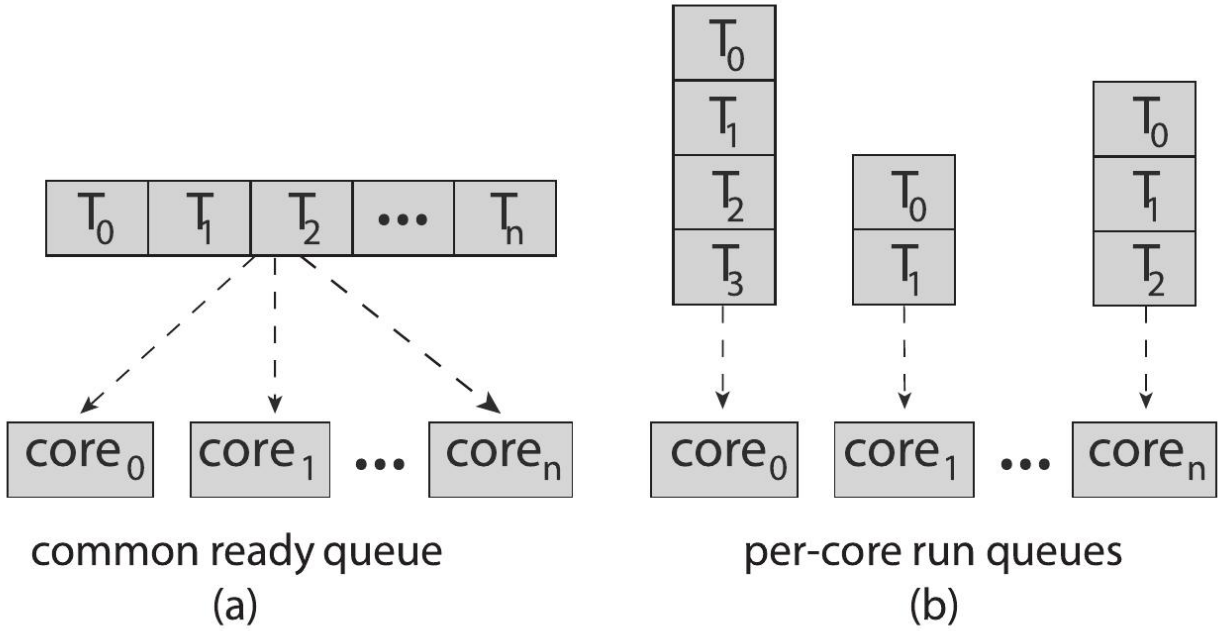


Figure 5.11 Organization of ready queues.

Kernel Parallelism under SMP



- Under SMP the kernel itself runs **in parallel** — several cores may be executing kernel code at the same instant.
- Consequences the kernel designer must handle:
  - Shared kernel data structures (ready queues, file tables) can be hit **concurrently** → race conditions.
  - Correctness needs **locking**; but locks serialise, so heavy locking destroys the parallelism you paid for.
  - Highly contended locks turn a shared structure into a **bottleneck** — exactly the shared-ready-queue problem
- The practical answer: **partition state per core**. Private run queues remove the lock from the hot path entirely.
- The kernel then only synchronises for the rarer cross-core operations — for example, migrating a thread during load balancing.

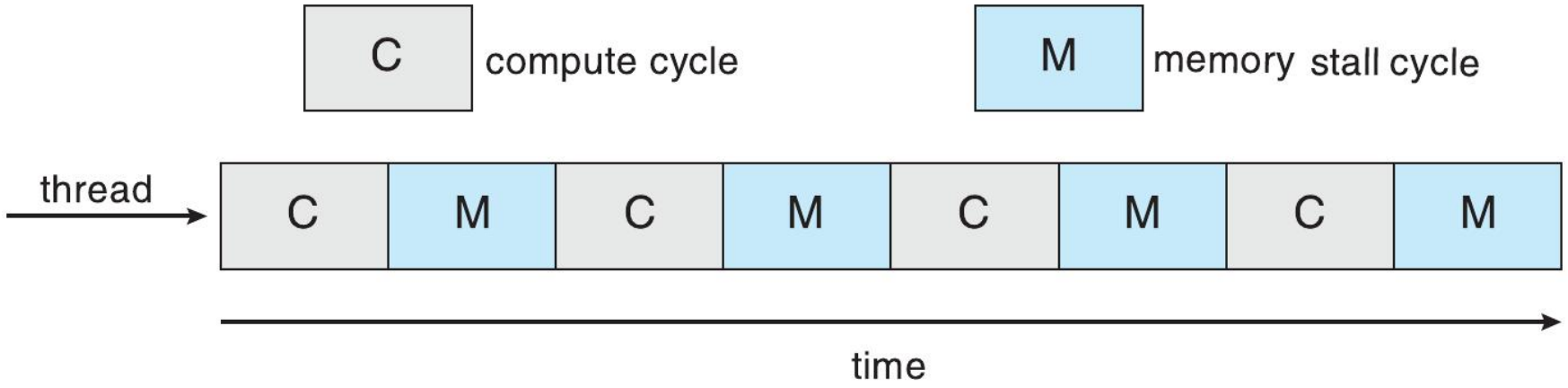
Multicore Processors



- Most contemporary hardware place multiple computing cores on the same physical chip resulting in a multicore processor.
- Each core maintains its own architectural state, so it appears to the operating system as a separate logical CPU.
- SMP systems built on multicore chips are **faster** and **consume less power** than systems giving each CPU its own chip.
- But multicore complicates scheduling.
- This situation, known as **memory stall** occurs mainly because processors operate at a speed much faster than memory. It also occurs because of **cache miss**- accessing data that are not in cache memory.

Memory Stall

- The thread alternates compute cycles (C) with memory stall cycles (M).
- In this scenario the processor can spend up to 50 percent of its time waiting for data from memory



Half the core is idle, but the core is not idle from the scheduler's point of view — the fix has to come from the hardware.

Multithreaded Processing Cores

- Hardware remedy: assign **two (or more) hardware threads to each core**.
- If one hardware thread stalls waiting for memory, the core switches to another thread—the stall cycles get filled.
- Each hardware thread keeps its own architectural state (instruction pointer, register set), so it appears to the OS as a logical CPU.

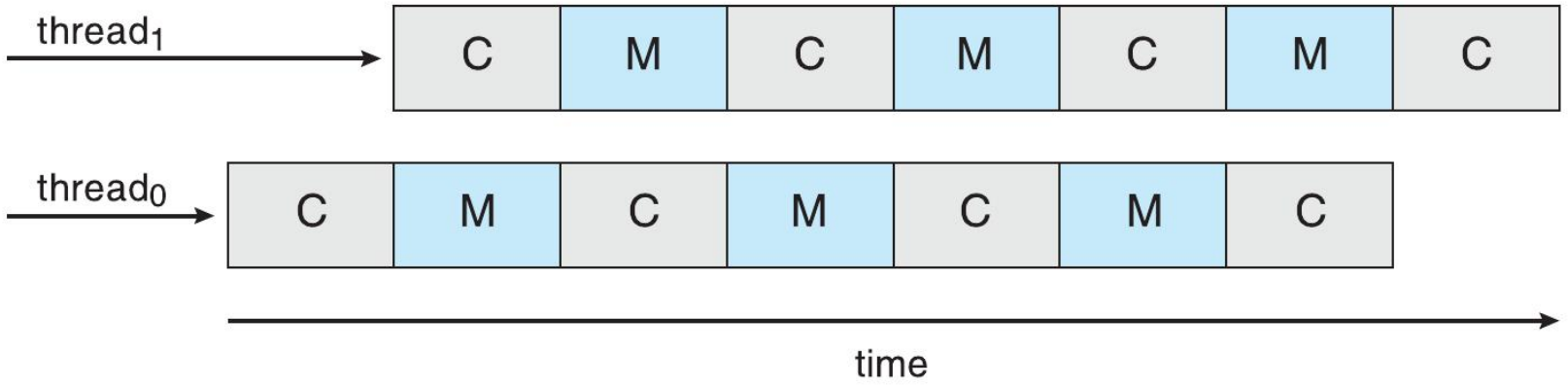
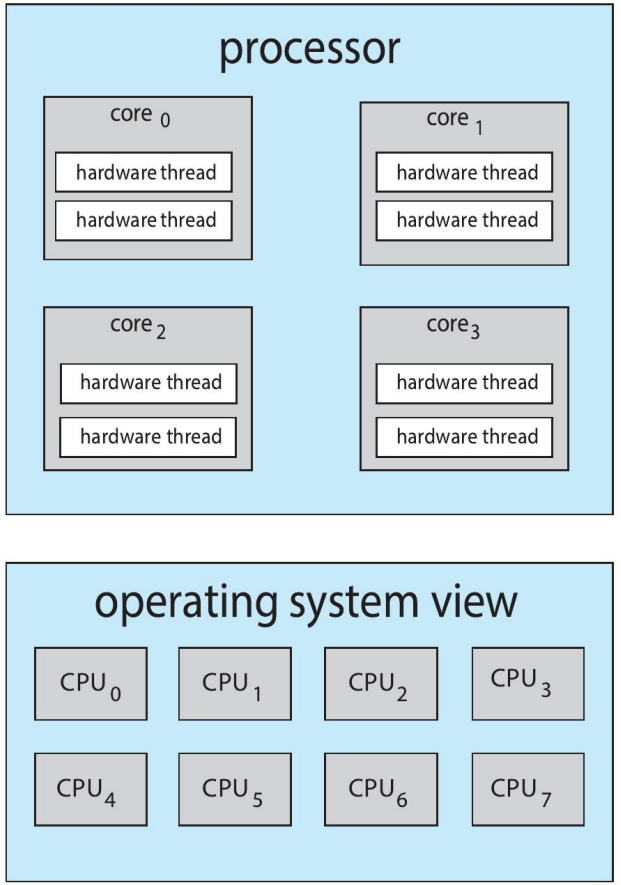


Figure 5.13 Multithreaded multicore system.

OPERATING SYSTEMS

Chip Multithreading

- This technique is called **chip multithreading (CMT)**.
- Here the processor has **four cores**, each with **two hardware threads**.
- From the operating system's point of view there are **eight logical CPUs**.
- The OS schedules software threads onto logical CPUs, not onto physical cores.
- Intel calls this **hyper-threading**, also known as **simultaneous multithreading (SMT)**.
- Contemporary Intel processors such as the i7 support 2 threads per core while the Oracle Sparc M&7supports 8 threads per core.



PES UNIVERSITY
CELEBRATING 50 YEARS

Load Balancing

- On SMP systems the workload must be kept balanced across all processors to get the benefit of having more than one. Otherwise some processors sit idle while others have high workloads.
- **Load balancing** attempts to keep the workload evenly distributed across all processors.
- There are 2 approaches to load balancing-

1. **Push migration**- A specific task periodically checks the load on each processor. If it finds an imbalance, it evenly distributes the load by moving (pushing) threads from overloaded processors to idle or less-busy ones.

2. **Pull migration**- Occurs when an idle processor pulls a waiting task from a busy processor

PES UNIVERSITY
CELEBRATING 50 YEARS

Processor Affinity

- While a thread runs on a processor, the data it accesses populate that processor's cache — **a warm cache.**
- Successive memory accesses are then satisfied from cache.
- If the thread **migrates** to another processor — say, due to load balancing:

  - The first processor's cache contents must be **invalidated.**

  - The second processor's cache must be **repopulated.**
- Because that cost is high, most SMP operating systems try to **avoid migrating a thread** and keep it on the same processor.
- This is **processor affinity**: a process has an affinity for the processor on which it is currently running.
- Two forms of processor affinity- **Soft affinity** and **Hard affinity**.

PES UNIVERSITY
CELEBRATING 50 YEARS

Processor Affinity



Soft affinity

- The OS has a policy of attempting to keep a process on the same processor but does not guarantee it.
- A process may still migrate between processors during load balancing.
- This is what Linux implements by default.

Hard affinity

- Provided through system calls.
- Lets a process specify a subset of processors on which it can run.
- Linux offers `sched_setaffinity()`, which lets a thread name the set of CPUs it is eligible to run on.



THANK YOU

Dr. Anitha K
Department of Computer Science and Engineering
anithak@pes.edu